

Optimasi Penjadwalan Ujian Universitas Menggunakan Algoritma Genetika

Husni Abdillah¹, Daffa Aulia Musyaffa Subyantoro¹, Qois Firosi¹, Ghiffari Bravia Hisham¹, and Naufal Ghifari Afdhala¹

Abstract—Penyusunan jadwal ujian universitas merupakan permasalahan optimasi kombinatorial yang kompleks karena harus menempatkan banyak mata kuliah pada slot waktu terbatas sambil menghindari bentrokan ujian mahasiswa. Penelitian ini menyajikan implementasi *Genetic Algorithm* (GA) untuk *University Examination Timetabling Problem* (UETP) dengan representasi kromosom satu dimensi, *tournament selection*, *one-point* dan *uniform crossover*, *swap* dan *move mutation*, elitisme, serta varian Hybrid GA yang memanfaatkan solusi Greedy dan mekanisme *repair* lokal. Kualitas solusi dihitung sebagai total penalti dari *hard constraint* dan *soft constraint*, sehingga nilai yang lebih rendah menunjukkan jadwal yang lebih baik. Eksperimen pada *dataset* simulasi berisi 2.340 mahasiswa, 150 mata kuliah, 6.220 relasi *enrollment*, 10 slot ujian, dan 10 ruang menunjukkan bahwa Hybrid GA menghasilkan penalti terbaik 1.114.518,5 pada eksperimen utama, lebih rendah daripada Pure GA 1.202.688,5 dan Greedy 1.221.557,5. Pada pengujian 30 kali, Hybrid GA juga memperoleh rata-rata penalti lebih rendah dan simpangan baku lebih kecil dibanding Pure GA. Hasil ini menunjukkan bahwa pendekatan evolusioner, terutama saat dikombinasikan dengan *seed* Greedy dan *repair* lokal, efektif untuk meningkatkan kualitas jadwal ujian dibanding *baseline* deterministik.

Index terms—Genetic Algorithm, Timetabling, UETP, Optimasi, Hybrid GA

I. PENDAHULUAN

A. Latar Belakang

Penyusunan jadwal ujian perguruan tinggi merupakan aktivitas akademik rutin yang memiliki tingkat kompleksitas tinggi. Setiap mata kuliah harus ditempatkan pada slot waktu tertentu, sementara mahasiswa dapat mengambil beberapa mata kuliah lintas semester, lintas program studi, atau mata kuliah umum universitas. Jika dua mata kuliah yang diambil mahasiswa yang sama dijadwalkan pada waktu yang sama, maka jadwal tersebut menghasilkan konflik yang tidak dapat diterima.

Masalah ini termasuk keluarga *University Examination Timetabling Problem* (UETP), yaitu masalah optimasi kom-

binatorial yang sulit diselesaikan secara eksak pada skala besar karena banyaknya kombinasi waktu, ruang, peserta, dan preferensi institusi [1], [2], [3]. Pendekatan manual biasanya bergantung pada pengalaman operator akademik dan sulit mengevaluasi seluruh kombinasi jadwal secara sistematis. Oleh karena itu, metode metaheuristik seperti *Genetic Algorithm* (GA) menjadi relevan karena mampu mengeksplorasi ruang solusi yang besar melalui proses evolusi populasi dan dapat dikombinasikan dengan perbaikan lokal [4], [5], [6].

Pada proyek ini, GA digunakan untuk membangun sistem penjadwalan ujian otomatis berbasis *dataset* CSV. Sistem tidak hanya mengevaluasi konflik mahasiswa, tetapi juga mempertimbangkan batasan jumlah ujian per hari, kapasitas ruang, blokir slot fakultas, penyebaran ujian, serta preferensi jarak antarujian. Hasil GA kemudian dibandingkan dengan algoritma Greedy sebagai *baseline*.

B. Rumusan Masalah

Rumusan masalah penelitian ini adalah sebagai berikut:

- 1) Bagaimana memodelkan solusi penjadwalan ujian universitas agar dapat diproses oleh *Genetic Algorithm*?
- 2) Bagaimana merancang fungsi *fitness* yang menggabungkan *hard constraint* dan *soft constraint* secara terukur?
- 3) Seberapa efektif Pure GA dan Hybrid GA dibandingkan *baseline* Greedy dalam menurunkan total penalti jadwal?
- 4) Bagaimana pengaruh parameter populasi dan mutasi terhadap konvergensi kualitas solusi?

C. Tujuan

Tujuan penelitian dan pengembangan sistem ini adalah:

- 1) Mengimplementasikan sistem penjadwalan ujian otomatis berbasis *Genetic Algorithm*.
- 2) Meminimalkan konflik jadwal ujian mahasiswa dan pelanggaran batasan tambahan.
- 3) Membandingkan performa Pure GA, Hybrid GA, dan *baseline* Greedy.
- 4) Menyediakan artefak hasil berupa jadwal ujian, riwayat *fitness*, statistik evaluasi, dan visualisasi eksperimen.

This work was not supported by any organization.

¹Abdillah, Subyantoro, Firosi, Hisham, Afdhala are with Departemen Ilmu Komputer, Institut Pertanian Bogor, Bogor, Jawa Barat, Indonesia, {ray25husni, daffaaulia, qoisfirosi, ghiffaribravia, naufalghifari}@apps.ipb.ac.id.

D. Manfaat

Sistem yang dikembangkan dapat membantu proses administrasi akademik dalam menghasilkan jadwal ujian yang lebih konsisten dan mudah dievaluasi. Dari sisi akademik, proyek ini juga menjadi studi kasus penerapan metaheuristik untuk masalah optimasi kombinatorial, lengkap dengan representasi solusi, operator evolusioner, fungsi *fitness*, *baseline*, serta analisis hasil eksperimen.

II. TINJAUAN PUSTAKA

A. University Examination Timetabling Problem (UETP)

University Examination Timetabling Problem adalah masalah penempatan ujian ke sejumlah periode waktu dan sumber daya tertentu dengan tujuan memenuhi batasan wajib dan mengoptimalkan preferensi kualitas jadwal. Dalam konteks universitas, batasan utama biasanya berupa larangan mahasiswa mengikuti dua ujian pada waktu yang sama. Selain itu, institusi dapat memiliki batasan kapasitas ruang, ketersediaan waktu tertentu, serta preferensi agar beban ujian mahasiswa tersebar dengan baik.

UETP bersifat NP-Hard karena jumlah kemungkinan kombinasi penempatan ujian meningkat sangat cepat seiring bertambahnya jumlah mata kuliah dan slot waktu. Jika terdapat N mata kuliah dan T slot waktu, maka ruang solusi kasar dapat mencapai T^N . Dengan 150 mata kuliah dan 10 slot, ruang solusi teoritisnya adalah 10^{150} , jauh melampaui kemampuan enumerasi penuh. Bahkan pada *benchmark* UETP yang lebih terstandar, pembuktian optimalitas dapat membutuhkan dekomposisi dan kombinasi pendekatan MIP serta *heuristic* khusus [7].

B. Algoritma Genetika

Genetic Algorithm adalah metode pencarian berbasis populasi yang meniru mekanisme seleksi alam. Setiap kandidat solusi direpresentasikan sebagai kromosom. Populasi solusi dievaluasi menggunakan fungsi *fitness*, kemudian individu yang lebih baik memiliki peluang lebih besar untuk mewariskan informasi genetiknya melalui operator seleksi, *crossover*, dan mutasi. Pada *examination timetabling*, GA banyak digunakan karena mudah dipadukan dengan representasi slot, objektif penalti, dan strategi *multi-objective* atau *local search* [4], [5], [8].

Pada masalah minimisasi seperti penjadwalan ujian, *fitness* dimaknai sebagai total penalti. Semakin kecil penalti, semakin baik kualitas jadwal. Siklus GA yang digunakan pada sistem ini terdiri atas inisialisasi populasi, evaluasi *fitness*, elitisme, pemilihan induk menggunakan *tournament selection*, *crossover*, mutasi, *optional repair* pada Hybrid GA, dan pembentukan generasi baru sampai batas generasi tercapai.

C. Representasi Kromosom

Representasi kromosom menentukan bagaimana suatu solusi potensial dimodelkan secara digital agar dapat dimanipulasi oleh operator genetika. Pada sistem ini, kromosom direpresentasikan sebagai sebuah larik satu dimensi (*1D array* atau `list[int]` pada Python).

Setiap elemen dalam larik tersebut memenuhi ketentuan sebagai berikut:

- 1) **Indeks larik** mewakili identitas mata kuliah. Urutan indeks dipetakan secara konsisten berdasarkan urutan data `courses.csv`.
- 2) **Nilai gen** pada indeks mewakili ID slot waktu yang dialokasikan untuk mata kuliah tersebut. Nilai berupa bilangan bulat dalam rentang $[1, T]$, dengan T sebagai jumlah slot waktu.

Keunggulan representasi ini adalah efisiensi memori dan kesederhanaan operator. Dibandingkan matriks mata kuliah terhadap slot waktu, representasi satu dimensi menghindari struktur *sparse* dan memudahkan *crossover* serta mutasi langsung pada tingkat gen. Pendekatan ini juga memastikan setiap mata kuliah selalu memiliki tepat satu slot ujian.

D. Batasan pada Penjadwalan

Constraint dalam sistem dibagi menjadi *hard constraint* dan *soft constraint*. *Hard constraint* diberi bobot penalti besar karena pelanggaran berdampak langsung terhadap kelayakan jadwal. *Soft constraint* diberi bobot lebih kecil karena berfungsi meningkatkan kenyamanan dan kualitas distribusi jadwal.

Hard constraint utama adalah konflik ujian mahasiswa, yaitu ketika seorang mahasiswa memiliki dua atau lebih ujian pada slot yang sama. Batasan tambahan yang juga dipenalti tinggi meliputi jumlah ujian maksimum per hari, kapasitas ruang ujian, dan blokir slot fakultas. *Soft constraint* meliputi ujian berturut-turut, terlalu banyak ujian dalam satu hari, ketidakseimbangan penyebaran slot, kedekatan mata kuliah inti semester yang sama, kedekatan mata kuliah berpeserta besar, *preferred gap*, dan penalti ujian Jumat sore. Pemisahan *hard* dan *soft constraint* seperti ini umum pada literatur *timetabling* karena kelayakan jadwal dan kualitas preferensi sering perlu dievaluasi secara bersamaan [9], [10], [11], [12].

E. Studi Literatur

Penelitian penjadwalan akademik mutakhir menempatkan *timetabling* sebagai masalah optimasi kombinatorial yang bergantung kuat pada konteks institusi. Survei Chen et al. dan Bashab et al. menunjukkan bahwa variasi *constraint*, *dataset*, dan tujuan optimasi membuat satu metode sulit berlaku universal untuk semua kampus [1], [2]. Abdipoor et al. juga menekankan bahwa metaheuristik dan *hybrid metaheuristic* menjadi pilihan dominan karena dapat menyeimbangkan eksplorasi ruang solusi dan pemenuhan *constraint* praktis [3].

Pendekatan eksak tetap penting sebagai pembanding dan sebagai cara memodelkan *constraint* secara formal. *Mixed-integer programming* dan *integer linear programming* telah digunakan untuk *course timetabling*, penugasan dosen, dan *examination timetabling* dengan *constraint* realistik seperti kapasitas ruang, preferensi pengajar, pemilihan hari, dan penggunaan ruang [9], [10], [11], [12], [13]. Namun, studi tersebut juga memperlihatkan bahwa model eksak cenderung membutuhkan reduksi masalah, *solver* khusus, atau batas waktu komputasi ketika skala dan kompleksitas *instance* meningkat.

Pada sisi *heuristic* dan *metaheuristic*, *local search*, *hyperheuristic*, dan algoritma populasi banyak dipakai untuk memperbaiki solusi secara iteratif. Bykov dan Petrovic menunjukkan efektivitas *Step Counting Hill Climbing* pada UETP, sedangkan Kheiri dan Keedwell serta Muklason et al. menunjukkan bahwa pemilihan *heuristic* dan operator *neighborhood* dapat berdampak besar pada performa pencarian [14], [15], [16]. Studi Nand et al. dan Siew et al. memperkuat temuan ini pada *dataset examination timetabling* modern dengan *Firefly Algorithm*, *Whale Optimization Algorithm*, dan komponen *local search* [17], [18].

Genetic Algorithm tetap relevan karena fleksibel dalam merepresentasikan kandidat solusi dan mudah dikombinasikan dengan *local search* atau *repair*. Son et al. menggunakan GA untuk *examination timetabling multi-objective*, Aslan menunjukkan bahwa *hybrid GA* dengan *local search* kompetitif pada UETP, dan Hafsa et al. menerapkan *evolutionary algorithms* pada *timetabling profesional multi-objective* [4], [5], [8]. Selain domain pendidikan, Fuladi dan Kim serta Al-Mudahka dan Alhamad memperlihatkan bahwa *scheduling/timetabling* berbasis optimasi juga digunakan pada konteks produksi dinamis dan kesehatan, sehingga desain fungsi penalti dan mekanisme perbaikan lokal menjadi prinsip yang dapat ditransfer lintas domain [6], [19]. Dalam perkembangan terbaru, Kallestad et al. menunjukkan arah *hyperheuristic* berbasis *deep reinforcement learning* untuk memilih operator pada masalah optimasi kombinatorial, yang relevan sebagai pengembangan lanjutan dari mekanisme *repair* atau *adaptive operator selection* [20].

III. METODOLOGI

A. Data dan Pemodelan Universitas

Data yang digunakan merupakan *dataset* simulasi universitas multijurusan. Data disimpan dalam format CSV agar mudah dibaca, diuji, dan diregenerasi. Struktur utama *dataset* terdiri atas mahasiswa, mata kuliah, *enrollment*, slot ujian, ruang, dan slot blokir fakultas.

Relasi *enrollment* menjadi sumber utama pembentukan konflik. Dua mata kuliah dianggap berkonflik jika terdapat minimal satu mahasiswa yang mengambil keduanya. Dengan

TABLE I. RINGKASAN DATA YANG DIGUNAKAN DALAM EKSPERIMEN

Berkas	Jumlah Data	Keterangan
students.csv	2.340	Mahasiswa beserta fakultas, departemen, dan semester
courses.csv	150	Mata kuliah umum, fakultas, dan departemen
enrollment.csv	6.220	Relasi mahasiswa terhadap mata kuliah
timeslots.csv	10	Slot ujian berdasarkan hari dan sesi
rooms.csv	10	Ruang ujian dan kapasitas
slot_blocks.csv	6	Larangan slot tertentu untuk fakultas

TABLE II. CONTOH PEMETAAN REPRESENTASI SOLUSI

Mata Kuliah	Slot Waktu
Kalkulus	3
Pancasila	5
Basis Data	2
Struktur Data	1
Kecerdasan Buatan	7

demikian, kualitas jadwal tidak hanya bergantung pada jumlah mata kuliah, tetapi juga pada kepadatan hubungan antar mata kuliah dalam data *enrollment*.

B. Prapemrosesan Data

Tahap prapemrosesan dimulai dari pembacaan CSV menjadi objek domain seperti *Student*, *Course*, *Enrollment*, dan *Timeslot*. Setelah itu, validator memastikan kolom wajib tersedia dan nilai numerik seperti semester, slot, hari, sesi, serta kapasitas dapat diproses.

Matriks konflik dibangun dengan mengelompokkan *enrollment* berdasarkan mahasiswa. Untuk setiap mahasiswa, semua pasangan mata kuliah yang diambil mahasiswa tersebut ditambahkan sebagai *edge* konflik. Hasil akhirnya berupa *adjacency list*, sehingga setiap mata kuliah memiliki daftar mata kuliah lain yang tidak ideal dijadwalkan pada slot yang sama.

C. Representasi Solusi

Sebagai perwujudan konkret dari rancangan kromosom, representasi solusi dalam kode diimplementasikan secara langsung untuk memetakan penugasan slot ujian.

Misalkan terdapat 5 mata kuliah dengan penugasan slot waktu berikut:

$$[3, 5, 2, 1, 7]$$

Secara struktural, kromosom tersebut diartikan sebagai pemetaan mata kuliah ke slot waktu seperti pada Table II.

Melalui pemetaan linier ini, indeks i dari larik merujuk pada elemen ke- i di dalam daftar mata kuliah terurut, dan nilai chromosome_i menentukan waktu pelaksanaan ujiannya. Representasi ini menjamin bahwa setiap mata kuliah pasti mendapatkan tepat satu slot waktu.

D. Inisialisasi Populasi

Proses pencarian solusi optimal diawali dengan pembentukan populasi awal yang terdiri atas sejumlah individu acak. Hal ini bertujuan memberikan titik awal pencarian yang tersebar di ruang solusi, meminimalkan risiko konvergensi dini, dan mempertahankan diversitas genetik.

Pada `population.py`, fungsi `initialize_population` menerima jumlah mata kuliah (N), jumlah slot waktu (T), ukuran populasi (P), dan *seed* opsional. Fungsi ini menghasilkan P individu, dengan setiap gen dipilih secara acak dari rentang slot valid:

$$\text{chromosome} = [R_1, R_2, \dots, R_N] \quad (1)$$

$$R_i \sim \text{Uniform}(1, T) \quad (2)$$

Pada Hybrid GA, populasi dapat diberi *seed* dari solusi Greedy. *Seed* ini menjadi titik awal yang sudah relatif layak, sementara individu acak tetap disertakan untuk menjaga eksplorasi.

E. Operator Algoritma Genetika

Operator genetika memproses generasi saat ini untuk menghasilkan generasi baru yang diharapkan memiliki kualitas solusi lebih baik. Implementasi operator diletakkan secara modular pada direktori `src/ga/`.

Seleksi. Sistem menggunakan *Tournament Selection* melalui fungsi `tournament_selection`. Sebanyak `tournament_size` individu dipilih secara acak dari populasi, kemudian individu dengan penalti terkecil menjadi pemenang turnamen dan digunakan sebagai induk. Nilai `tournament_size` mengatur tekanan seleksi: semakin besar nilainya, semakin kuat preferensi terhadap individu terbaik, tetapi diversitas populasi dapat menurun lebih cepat.

Crossover. Operator *crossover* menggabungkan dua induk untuk menghasilkan dua anak. Sistem menggunakan *one-point crossover*, yaitu memilih satu titik potong lalu menukar segmen kromosom setelah titik tersebut, dan *uniform crossover*, yaitu mengevaluasi setiap posisi gen dan menukar gen antarinduk berdasarkan peluang `swap_prob`. Pada `engine.py`, tipe *crossover* dipilih secara acak dengan peluang seimbang setelah pasangan induk lolos probabilitas `crossover_rate`.

Mutasi. Mutasi mengenalkan variasi genetik baru agar pencarian tidak mudah terjebak pada optimum lokal. Sistem menggunakan *swap mutation*, yaitu menukar dua gen secara acak, dan *move mutation*, yaitu mengganti nilai gen dengan slot baru berdasarkan probabilitas `mutation_rate`. *Move*

TABLE III. SIMBOL DAN BOBOT BATASAN DALAM FUNGSI *FITNESS*

Simbol	Bobot	Komponen
H	1000	Konflik mahasiswa
C	10	Ujian berturut-turut
D	5	Terlalu banyak ujian harian
S	30	Ketimpangan distribusi slot
M	3	Mata kuliah inti terlalu dekat
E	2	Mata kuliah berpeserta besar
G	1	Preferred gap
F	5	Jumat sesi 3
X	200	Maksimum ujian per hari
R	300	Kapasitas ruang
B	250	Slot blokir fakultas

mutation penting pada UETP karena satu perubahan slot dapat langsung mengurangi konflik besar pada mata kuliah tertentu.

Elitisme dan repair. Elitisme mempertahankan sejumlah `elite_count` individu terbaik secara utuh ke generasi berikutnya. Hybrid GA menambahkan *repair* lokal ketika `enable_repair` aktif. *Repair* dijalankan secara probabilistik pada anak hasil evolusi untuk mengevaluasi alternatif slot yang menurunkan penalti.

F. Pemodelan Batasan

Fungsi objektif sistem adalah minimisasi total penalti. Nilai *fitness* dihitung sebagai penjumlahan berbobot dari pelanggaran *constraint*:

$$P = 1000H + 10C + 5D + 30S + 3M + 2E + G + 5F + 200X + 300R + 250B \quad (3)$$

dengan P adalah total penalti. Definisi simbol ditunjukkan pada Table III.

G. Fungsi Fitness

Fungsi *fitness* berada pada `src/fitness/fitness.py` dan mengembalikan total penalti bertipe `float`. Semakin kecil nilai *fitness*, semakin baik solusi. Komponen *hard constraint* dihitung dari daftar mata kuliah setiap mahasiswa dengan menghitung pasangan ujian yang berada pada slot sama. Komponen *soft constraint* dihitung dengan memetakan slot menjadi pasangan hari-sesi sehingga sistem dapat mengevaluasi beban ujian per hari, kedekatan sesi, dan distribusi ujian.

Fungsi *fitness* juga menggunakan *cache* data global untuk mempercepat evaluasi berulang. Hal ini penting karena GA mengevaluasi banyak kromosom pada setiap generasi. Dengan parameter populasi 50 dan 50 generasi, sedikitnya terdapat ribuan evaluasi *fitness* dalam satu eksperimen.

H. Algoritma Greedy Baseline

Baseline Greedy digunakan sebagai pembanding deterministik. Algoritma ini mengurutkan mata kuliah berdasarkan derajat konflik, yaitu jumlah mata kuliah lain yang berbenturan dengannya. Mata kuliah dengan konflik terbanyak dijadwalkan lebih awal. Untuk setiap mata kuliah, Greedy mencoba semua

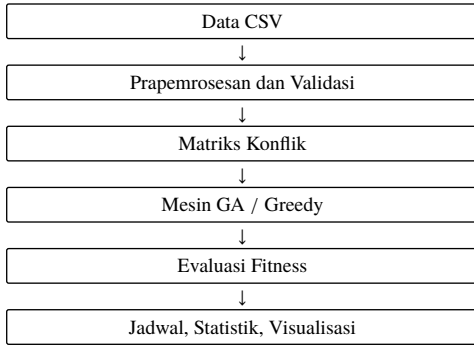


Fig. 1. Flowchart arsitektur sistem penjadwalan ujian

slot yang tersedia dan memilih slot yang menghasilkan penalti terkecil pada kondisi saat itu.

Pendekatan ini cepat dan mudah dipahami, tetapi memiliki kelemahan utama: keputusan awal tidak direvisi secara global. Jika suatu mata kuliah sudah ditempatkan pada slot tertentu, keputusan tersebut dapat membatasi pilihan mata kuliah berikutnya. GA mengatasi kelemahan ini dengan mempertahankan banyak kandidat solusi dan memperbaikinya lintas generasi.

I. Arsitektur Sistem

Arsitektur sistem dirancang modular agar prapemrosesan, GA, *fitness*, evaluasi, dan visualisasi dapat diuji secara terpisah. Alur data utama ditunjukkan pada Fig. 1.

Modul prapemrosesan bertanggung jawab membaca dan memvalidasi data. Modul *ga* menjalankan proses evolusi. Modul *fitness* menghitung penalti. Modul *evaluation* menyediakan *baseline* Greedy dan *benchmark*. Modul *visualization* menghasilkan tabel jadwal serta grafik performa.

IV. HASIL DAN PEMBAHASAN

A. Spesifikasi Eksperimen

Eksperimen utama menggunakan konfigurasi GA pada Table IV. Selain eksperimen utama, dilakukan analisis sensitivitas terhadap ukuran populasi dan *mutation_rate*, pengujian statistik 30 *run*, serta profil waktu eksekusi.

B. Hasil Algoritma Genetika

Hasil utama pada *outputs/statistics.json* menunjukkan bahwa Hybrid GA memperoleh total penalti terendah. Pure GA masih lebih baik daripada Greedy, tetapi perbaikannya lebih kecil dibanding Hybrid GA. Ringkasan hasil ditunjukkan pada Table V.

Dibanding Greedy, Pure GA menurunkan *fitness* sebesar 18.869 poin atau sekitar 1,54%. Hybrid GA menurunkan *fitness* sebesar 107.039 poin atau sekitar 8,77%. Hybrid GA juga menghilangkan pelanggaran terlalu banyak ujian harian pada hasil utama, sedangkan Greedy masih memiliki 25 pelanggaran dan Pure GA 18 pelanggaran.

TABLE IV. KONFIGURASI UTAMA EKSPERIMEN

Parameter	Nilai
Jumlah mata kuliah	150
Jumlah slot ujian	10
Ukuran populasi	50
Generasi maksimum	50
Laju <i>crossover</i>	0,8
Laju mutasi	0,1
Ukuran turnamen	5
Jumlah elit	2
Probabilitas <i>repair</i> Hybrid GA	0,20

TABLE V. PERBANDINGAN HASIL EKSPERIMEN UTAMA

Metode	Waktu (s)	Penalti	Konflik	Ujian Harian	Sebaran
Greedy	14,265	1.221.557,5	1.216	25	4,25
Pure GA	21,578	1.202.688,5	1.198	18	22,25
Hybrid GA	19,078	1.114.518,5	1.114	0	7,25

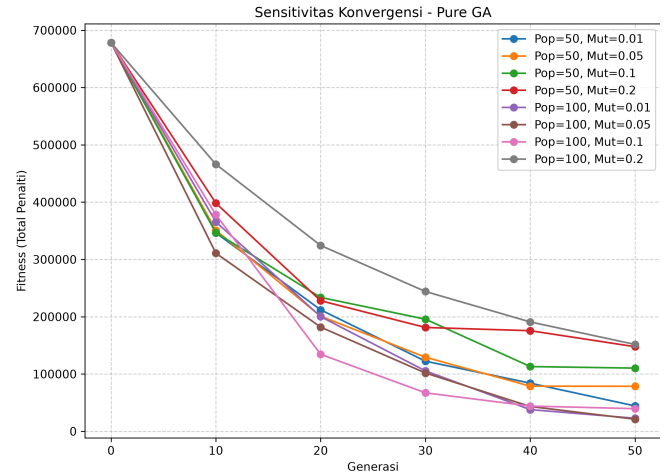


Fig. 2. Konvergensi sensitivitas parameter pada Pure GA

C. Visualisasi Konvergensi

Riwayat *fitness* pada *outputs/fitness_history.csv* menunjukkan penurunan penalti sepanjang generasi. Pure GA dimulai dari 2.159.201,5 pada generasi 0 dan mencapai 1.202.688,5 pada generasi akhir. Hybrid GA dimulai dari 1.221.557,5 karena menggunakan *seed* Greedy, lalu turun ke 1.114.518,5.

Pada Fig. 2 dan Fig. 3, Pure GA tampak lebih bergantung pada populasi dan *mutation_rate*. Populasi 100 dengan *mutation_rate* 0,05 menghasilkan *fitness* terbaik pada analisis sensitivitas, sedangkan *mutation_rate* 0,2 cenderung lebih buruk karena perubahan gen terlalu agresif. Hybrid GA memiliki nilai awal jauh lebih rendah karena *seed* Greedy dan menghasilkan kurva yang lebih stabil.

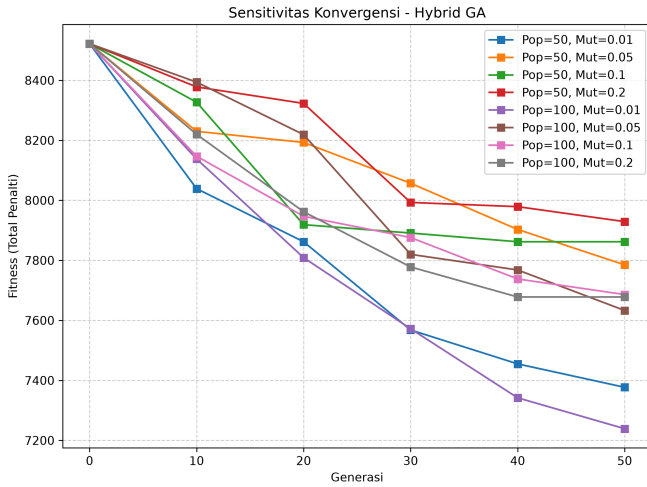


Fig. 3. Konvergensi sensitivitas parameter pada Hybrid GA

TABLE VI. CONTOH KELUARAN JADWAL UJIAN TERBAIK

Kode	Mata Kuliah	Slot	Hari	Sesi
GEN101	Pancasila	4	4	1
GEN102	Pendidikan Agama	2	2	1
GEN103	Bahasa Indonesia	4	4	1
GEN104	Bahasa Inggris	3	3	1
GEN105	Kewirausahaan	1	1	1
KOM201	Pemrograman Dasar	3	3	1
KOM401	Struktur Data	4	4	1
KOM603	Kecerdasan Buatan	1	1	1

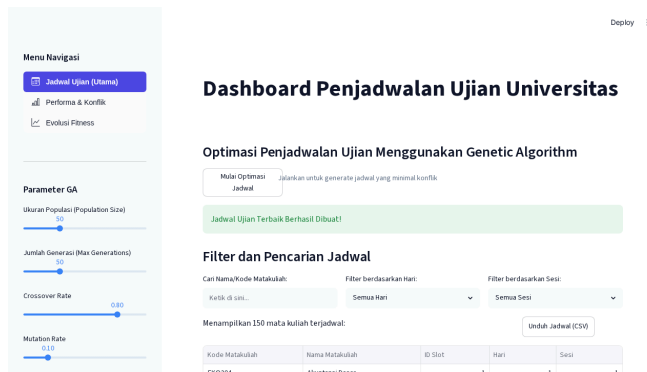


Fig. 4. Tampilan dashboard Streamlit untuk jadwal ujian

D. Visualisasi Jadwal

Jadwal terbaik diekspor ke `outputs/schedule.csv` dan berisi 150 mata kuliah. Setiap baris memuat kode mata kuliah, nama mata kuliah, slot, hari, dan sesi. Contoh sebagian jadwal ditampilkan pada Table VI.

Selain keluaran CSV, sistem menyediakan antarmuka Streamlit untuk menjalankan optimasi, mengatur parameter GA, memfilter jadwal, serta mengunduh hasil. Tampilan halaman utama aplikasi ditunjukkan pada Fig. 4.

TABLE VII. RINGKASAN PENGUJIAN STATISTIK 30 RUN

Metode	Terbaik	Terburuk	Rata-rata	Simp. Baku
Pure GA	1.048.819,5	1.203.468,5	1.126.067,87	39.893,37
Hybrid GA	1.062.901,5	1.122.514,5	1.088.218,17	17.692,77

E. Perbandingan dengan Greedy

Greedy memiliki keunggulan pada kesederhanaan dan determinisme. Pada eksperimen utama, waktu Greedy adalah 14,265 detik, lebih cepat daripada Pure GA. Namun, kualitas jadwal Greedy lebih rendah karena total penalti dan pelanggaran *hard constraint* lebih tinggi.

Perbandingan dengan Greedy juga bergantung pada jumlah generasi yang diberikan kepada GA. Pada profil kompleksitas waktu, Greedy menghasilkan solusi tetap karena tidak memiliki parameter generasi, sedangkan kualitas GA berubah seiring bertambahnya generasi. Pure GA membutuhkan generasi lebih banyak untuk keluar dari populasi awal yang buruk. Sebaliknya, Hybrid GA dapat melampaui Greedy lebih cepat karena menggunakan *seed* Greedy dan *repair* lokal, tetapi biaya waktunya meningkat ketika jumlah generasi diperbesar.

Pengujian statistik 30 kali pada `outputs/statistical_significance.json` memperlihatkan pola yang lebih kuat. *Baseline* Greedy memiliki *fitness* 1.218.831,5. Pure GA memiliki rata-rata *fitness* 1.126.067,87 dengan simpangan baku 39.893,37, sedangkan Hybrid GA memiliki rata-rata *fitness* 1.088.218,17 dengan simpangan baku 17.692,77. Artinya, Hybrid GA tidak hanya menghasilkan rata-rata penalti yang lebih rendah, tetapi juga lebih stabil antar-run.

F. Analisis Hasil

Hasil eksperimen memperlihatkan tiga temuan utama. Pertama, representasi kromosom satu dimensi cukup efektif karena semua operator dapat bekerja langsung pada slot mata kuliah tanpa perlu memperbaiki struktur solusi yang tidak valid. Setiap individu selalu merepresentasikan jadwal lengkap.

Kedua, Pure GA mampu memperbaiki solusi melalui eksplorasi populasi, tetapi membutuhkan generasi yang cukup untuk mendekati kualitas Greedy. Hal ini terlihat pada riwayat *fitness* yang turun bertahap dari nilai awal yang tinggi. Ketika ruang solusi sangat besar, populasi acak awal sering memiliki konflik tinggi.

Ketiga, Hybrid GA memberikan *trade-off* terbaik pada eksperimen ini. *Seed* Greedy menyediakan solusi awal yang sudah cukup baik, sedangkan operator GA dan *repair* lokal memperbaiki kelemahan keputusan Greedy yang terlalu lokal. Waktu eksekusi Hybrid GA memang dapat meningkat pada profil generasi yang lebih panjang, tetapi peningkatan kualitas dan stabilitas solusi membuatnya lebih sesuai untuk skenario penjadwalan akademik yang tidak harus dilakukan secara *real-time*.

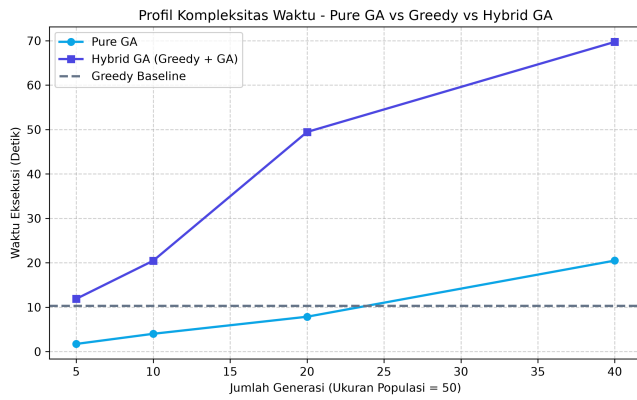


Fig. 5. Profil waktu eksekusi terhadap jumlah generasi

V. KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan implementasi dan eksperimen, dapat disimpulkan bahwa:

- 1) Masalah penjadwalan ujian dapat dimodelkan secara efektif menggunakan kromosom satu dimensi yang memetakan mata kuliah ke slot ujian.
- 2) Fungsi *fitness* berbasis penalti berbobot mampu menggabungkan konflik mahasiswa, kapasitas ruang, slot blokir, dan *soft constraint* kualitas jadwal.
- 3) Pure GA berhasil menurunkan penalti dibanding Greedy pada eksperimen utama, dari 1.221.557,5 menjadi 1.202.688,5.
- 4) Hybrid GA memberikan hasil terbaik dengan penalti 1.114.518,5 pada eksperimen utama dan rata-rata 1.088.218,17 pada 30 *run*, lebih rendah serta lebih stabil daripada Pure GA.
- 5) *Seed Greedy* dan *repair* lokal terbukti membantu GA memulai dari solusi yang lebih baik serta mengurangi risiko pencarian terlalu lama pada area solusi buruk.

B. Saran

Pengembangan berikutnya dapat diarahkan pada beberapa aspek. Pertama, sistem dapat menggunakan *adaptive mutation* agar laju mutasi berubah berdasarkan stagnasi *fitness*. Kedua, *repair* lokal dapat dibuat lebih selektif dengan memprioritaskan mata kuliah penyebab konflik terbesar. Ketiga, *dataset* riil dari sistem akademik dapat digunakan untuk menguji ketahanan model terhadap distribusi *enrollment* yang lebih kompleks. Keempat, optimasi dapat diperluas ke *multi-objective optimization* agar konflik, kenyamanan mahasiswa, pemakaian ruang, dan preferensi institusi dapat dianalisis sebagai objektif terpisah.

REFERENCES

- [1] M. C. Chen, S. N. Sze, S. L. Goh, N. R. Sabar, and G. Kendall, "A Survey of University Course Timetabling Problem: Perspectives, Trends and Opportunities," *IEEE Access*, vol. 9, pp. 106515–106529, 2021, doi: 10.1109/ACCESS.2021.3100613.
- [2] A. Bashab *et al.*, "Optimization Techniques in University Timetabling Problem: Constraints, Methodologies, Benchmarks, and Open Issues," *Computers, Materials and Continua*, vol. 74, no. 3, 2023, doi: 10.32604/cmc.2023.034051.
- [3] S. Abdipoor, R. Yaakob, S. L. Goh, and S. Abdullah, "Meta-Heuristic Approaches for the University Course Timetabling Problem," *Intelligent Systems with Applications*, vol. 19, p. 200253, 2023, doi: 10.1016/j.iswa.2023.200253.
- [4] N. T. Son, J. B. Jaafar, I. A. Aziz, H. G. Nguyen, and A. N. Bui, "Genetic Algorithm for Solving Multi-Objective Optimization in Examination Timetabling Problem," *International Journal of Emerging Technologies in Learning*, vol. 16, no. 11, pp. 4–24, 2021, doi: 10.3991/ijet.v16i11.21017.
- [5] A. Aslan, "Some Experiences with Hybrid Genetic Algorithms in Solving the Uncapacitated Examination Timetabling Problem," *arXiv preprint arXiv:2306.00534*, 2023.
- [6] S. K. Fuladi and C.-S. Kim, "Dynamic Events in the Flexible Job-Shop Scheduling Problem: Rescheduling with a Hybrid Metaheuristic Algorithm," *Algorithms*, vol. 17, no. 4, p. 142, 2024, doi: 10.3390/a17040142.
- [7] A. Dimitis, C. Gogos, C. Valouxis, V. Nastos, and P. Alefragis, "A Proven Optimal Result for a Benchmark Instance of the Uncapacitated Examination Timetabling Problem," *Journal of Scheduling*, vol. 28, pp. 183–194, 2025, doi: 10.1007/s10951-024-00805-0.
- [8] M. Hafsa, P. Wattebled, J. Jacques, and L. Jourdan, "Solving a Multiobjective Professional Timetabling Problem Using Evolutionary Algorithms at Mandarine Academy," *International Transactions in Operational Research*, vol. 32, pp. 244–269, 2025, doi: 10.1111/itor.13276.
- [9] H. Algethami and W. Laesanklang, "A Mathematical Model for Course Timetabling Problem With Faculty-Course Assignment Constraints," *IEEE Access*, vol. 9, pp. 111666–111682, 2021, doi: 10.1109/ACCESS.2021.3103495.
- [10] N. M. Arratia-Martinez, C. Maya-Padron, and P. A. Avila-Torres, "University Course Timetabling Problem with Professor Assignment," *Mathematical Problems in Engineering*, vol. 2021, p. 6617177, 2021, doi: 10.1155/2021/6617177.
- [11] M. Mokhtari, M. Vaziri Sarashk, M. Asadpour, N. Saeidi, and O. Boyer, "Developing a Model for the University Course Timetabling Problem: A Case Study," *Complexity*, vol. 2021, p. 9940866, 2021, doi: 10.1155/2021/9940866.
- [12] E. Rappos, E. Thiemard, S. Robert, and J.-F. Heche, "A Mixed-Integer Programming Approach for Solving University Course Timetabling Problems," *Journal of Scheduling*, vol. 25, pp. 391–404, 2022, doi: 10.1007/s10951-021-00715-5.
- [13] T. Laisupannawong and S. Sumetthapiwat, "An Integer Linear Programming Model for the Examination Timetabling Problem: A Case Study of a University in Thailand," *Advances in Operations Research*, vol. 2024, p. 6636563, 2024, doi: 10.1155/2024/6636563.
- [14] Y. Bykov and S. Petrovic, "A Step Counting Hill Climbing Algorithm Applied to University Examination Timetabling," *Journal of Scheduling*, vol. 19, pp. 479–492, 2016, doi: 10.1007/s10951-016-0469-x.
- [15] A. Kheiri and E. Keedwell, "A Hidden Markov Model Approach to the Problem of Heuristic Selection in Hyper-Heuristics with a Case Study in High School Timetabling Problems," *Evolutionary Computation*, 2016, doi: 10.1162/EVCO_a_00186.
- [16] A. Muklasun, Y. Tendio, H. A. Depari, M. A. Nuriman, and I. G. A. Premananda, "The Performance Analysis of Hyper-Heuristics Algorithms over Examination Timetabling Problems," *IAES International Journal of Artificial Intelligence*, vol. 13, no. 2, pp. 2155–2164, 2024, doi: 10.11591/ijai.v13.i2.pp2155-2164.
- [17] R. Nand, E. Reddy, K. Chaudhary, and B. Sharma, "Preference-Based Stepping Ahead Firefly Algorithm for Solving Real-World Uncapacitated Examination Timetabling Problem," *IEEE Access*, vol. 12, pp. 24685–24702, 2024, doi: 10.1109/ACCESS.2024.3365734.
- [18] E. S. K. Siew, S. N. Sze, and S. L. Goh, "Optimizing Decentralized Exam Timetabling with a Discrete Whale Optimization Algorithm," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 1, pp. 257–268, 2025, doi: 10.14569/IIACSA.2025.0160125.

- [19] I. Al-Mudahka and R. Alhamad, "On a Timetabling Problem in the Health Care System," *RAIRO - Operations Research*, vol. 56, pp. 4347–4362, 2022, doi: 10.1051/ro/2022182.
- [20] J. Kallestad, R. Hasibi, A. Hemmati, and K. Sorensen, "A General Deep Reinforcement Learning Hyperheuristic Framework for Solving Combinatorial Optimization Problems," *European Journal of Operational Research*, vol. 309, no. 2, pp. 446–468, 2023, doi: 10.1016/j.ejor.2023.01.017.